

problem:
目前35份格式太一致，因為現在使用mock data，有需要生成很貼近真實的各種情況嗎？
2026-05-05

若要各種情況:
應該用不同的prompt，還是同一個prompt但設定生成的range(有格式~沒格式、很多內容~很少內容)
格式可以一致嗎?

加入persona，有好有壞，有各種情況

審查系統設計文件 v2

Mock 資料說明

系統以 35 筆 AI 生成的虛構申請者文件 作為測試資料，不使用任何真實個人資料。

生成方式

Mock 資料由兩階段 Gemini API 自動生成 (mock_data/generate_mock.py)：

- 1. **Phase 1**: 依分布設定 (mock_data/prompt/distribution.txt) 生成 35 筆申請者屬性摘要
- 2. **Phase 2**: 依每筆屬性生成完整推甄文件 (自傳 + 讀書計畫 + 其他加分文件)

資料分布設計

維度	分布
學校	台大/清大/交大/成大 40%、中央/中山/中興/中正 40%、其他 20%
主要領域	軟體 60%、硬體 20%、韌體 10%、跨域 10%
研究子領域	資安、AI/ML、EDA、分散式系統、生醫、NLP、編譯器等 30+ 方向

high_level_mock.txt
distribution.txt

檔案對應 & High level mock data

角色	路徑
生成腳本	mock_data/generate_mock.py
分布設定	mock_data/prompt/distribution.txt
原始輸入文件	mock_data/mock_01.txt ~ mock_35.txt
Pipeline 輸出	output_mock/mock_01.json ~ mock_35.json

技術架構

- **後端**: Node.js + TypeScript (ts-node)
- **LLM**: Gemini API (文字切割、標註、Sub-criteria 命名、Hashtag 篩選)
- **Embedding**: Gemini Embedding API (gemini-embedding-2)
- **分群**: K-Means + GMM (本地計算，無 Vector DB)
- **視覺化**: UMAP + Plotly.js (輸出靜態 HTML)
- **Criteria**: 由校務會議預先定義，共 4 個，系統不自動探索

預定義 Criteria

編號	名稱
C1	學術根基與跨域修課表現

編號	名稱
C2	專題實作與技術應用能力
C3	問題解決與批判性思考
C4	專業傳遞與自我成長規劃

輸入文件類型

- 自傳
- 讀書計畫
- 其他加分文件（得獎紀錄、課堂作品、專題報告等）

Pipeline 流程

步驟 1 — Idea Unit 分割

- 輸入：申請者文件
- 用 LLM 依語意切割成 Idea Units，每個語意完整不可再分
- 條列式活動紀錄：每條切為獨立 Idea Unit
- 輸出：`candidate.json`，含 `idea_units[]`

對應檔案：

類型	檔案
執行指令	<code>scripts/runStep01.ts</code>
前處理邏輯	<code>src/services/preprocessor.ts</code>
分割邏輯	<code>src/services/ideaUnitSplitter.ts</code>
Prompt	<code>src/prompts/ideaUnitSplit.ts</code>

指令：`npm run pipeline / npm run pipeline:mock`

步驟 2 — Idea Unit 標註

對每個 Idea Unit，LLM 執行三件事：

1. 映射到 **1-2 個 Criteria**
 2. 對每個映射到的 Criterion，自由命名一個 **raw sub-criteria label**
 3. 提取 **hashtag 關鍵字**（全體共用，不分 Criterion）
- `mock_06.json`

輸出欄位（寫入 IdeaUnit）：

- `criteria: CriterionId[]`
- `sub_criteria_map: Partial<Record<CriterionId, string>>`
- `hashtags: string[]`

對應檔案：

類型	檔案
執行指令	scripts/runStep02.ts
標註邏輯	src/services/annotator.ts
Prompt	src/prompts/annotate.ts

指令: npm run step2 / npm run step2:mock

步驟 3 — Sub-criteria 收斂

standard_distionary.json

針對每個 Criterion 分別處理（全部候選人）：

- 1. 收集該 Criterion 下所有候選人的 raw sub-criteria labels
- 2. 對 labels 做 Gemini Embedding，結果快取至 embeddings_cache.json（按 model name 失效）
- 3. **K-Means**（K 範圍 3–10，Silhouette Score 選最佳 K）
- 4. Silhouette Score 低於 0 的點標為 outlier，歸入「其他面向」 bucket
- 5. 單一 LLM call 對所有群同時命名（確保 mutual exclusivity），產出標準 sub-criteria 名稱
- 6. 增量存檔：每個 Criterion 完成後立即寫入 standard_dictionary.json（支援中斷續跑）

輸出: standard_dictionary.json，每個 Criterion 底下若干標準 Sub-criteria（含 id、name、raw_labels[]）

對應檔案：

類型	檔案
執行指令	scripts/runStep03.ts
主要邏輯	src/services/subCriteriaClusterer.ts
Embedding 快取	src/services/embedder.ts
命名 Prompt	src/prompts/clusterNaming.ts
K-Means 演算法	src/utils/kmeans.ts
GMM 演算法	src/utils/gmm.ts

指令: npm run step3 / npm run step3:mock

步驟 4 — Relabeling

mock_06.json

- 讀取 standard_dictionary.json，建立 rawLabel → standardName 反查表
- 為每個 Idea Unit 新增 standard_sub_criteria_map（保留原始 sub_criteria_map 不動）
- Skip 邏輯：已有 standard_sub_criteria_map 的 Idea Unit 不重複處理

輸出欄位（新增至 IdeaUnit）：

- standard_sub_criteria_map: Partial<Record<CriterionId, string>>

對應檔案：

類型	檔案
執行指令	<code>scripts/runStep04.ts</code>
Relabeling 邏輯	<code>src/services/relabeler.ts</code>

不需要 LLM，純字典查表。

指令：`npm run step4 / npm run step4:mock`

步驟 5 — 全量處理（略過）

因所有候選人已在步驟 2 完成標註，步驟 5 不需要。

沒有對應檔案：原設計中步驟 5 是針對「前 30 份以外的剩餘文件」補跑步驟 2 的流程。由於實作時所有候選人在步驟 2 就一起處理（沒有 30 份的限制），本步驟自然消失。

步驟 6 — 候選人特徵向量建構

1. 從 `standard_dictionary.json` 建立有序 sub-criteria index (C1 → C4, others 置後)
2. 對每個候選人，統計 `standard_sub_criteria_map` 中各 sub-criteria 的出現次數 → 計數向量
3. **L2 正規化**（消除文件量多寡導致的計數偏差，保留各 sub-criteria 的比例分布）
4. 同時統計各 Criterion 的 Idea Unit 數量 → `radar_chart_data`
5. Skip 邏輯：已有 `feature_vector` 的候選人不重處理（可用 `FORCE=1` 強制重建）

輸出欄位（寫入 Candidate）：

- `feature_vector`: `number[]` (42 維, L2 正規化)
- `radar_chart_data`: `Record<CriterionId, number>`

對應檔案：

類型	檔案
執行指令	<code>scripts/runStep06.ts</code>
主要邏輯	<code>src/services/featureBuilder.ts</code>

不需要 LLM，純計算。

指令：`npm run step6 / npm run step6:mock`

步驟 7 — 候選人分群

[UMAP.html](#)

1. 讀取所有候選人 `feature_vector`
2. 同時試 **K-Means** (Silhouette Score 選最佳 K) 與 **GMM** (BIC 選最佳 K)，K 範圍 2-6
3. 選 Silhouette Score 較高的演算法與 K 值作為最終結果
4. **Medoid 選取**：每個 cluster 內平均歐氏距離最小的成員，標記 `is_medoid: true`

5. **UMAP 降至 2D**，以 Plotly.js 產生互動式 HTML (`umap.html`)
6. 結果寫回每個候選人檔案，並輸出 `clusters.json`

輸出欄位（寫入 Candidate）：

- `cluster_id`: number
- `is_medoid`: boolean

輸出檔案：

- `clusters.json` (ClusterSummary, 含每個 cluster 的成員與 medoid)
- `umap.html` (互動式散點圖)

對應檔案：

類型	檔案
執行指令	<code>scripts/runStep07.ts</code>
主要邏輯	<code>src/services/candidateClusterer.ts</code>
K-Means 演算法	<code>src/utils/kmeans.ts</code>
GMM 演算法	<code>src/utils/gmm.ts</code>

不需要 LLM，純計算。

指令: `npm run step7 / npm run step7:mock`

步驟 8 — 群體命名

clusters.json

1. 對每個 cluster，統計成員中最高頻的 **top-5 sub-criteria** 與 **top-10 hashtags**
2. 將統計結果交給 LLM，產出 **4-8 字中文群名**
3. Skip 邏輯：已有 `cluster_name` 的 cluster 不重複命名
4. 將名稱與統計寫回 `clusters.json`，並寫入每個成員的 `cluster_name`

輸出欄位（寫入 Candidate）：

- `cluster_name`: string

對應檔案：

類型	檔案
執行指令	<code>scripts/runStep08.ts</code>
主要邏輯	<code>src/services/clusterNamer.ts</code>
Prompt	<code>src/prompts/clusterGroupNaming.ts</code>

指令: `npm run step8 / npm run step8:mock`

步驟 9 — Distinctive Hashtag 篩選

mock_06.json

對每個候選人，用 LLM 從其所有 hashtag（去重後）中選出 3–4 個最能代表其亮點的標籤。

Prompt 選擇標準：

- 技術專長：具體技術領域、特定研究主題、有鑑別力的工具或方法
- 特殊經歷：競賽、論文發表、專利、開源貢獻、值得注意的實習、突出跨域成就
- 排除：學校名稱、一般社團、運動、音樂才藝、個人特質、太通用的技術詞

格式規範：單一概念、不超過 15 字、不加括號補充、不用頓號連接多項、要具體不能只寫類別名

Skip 邏輯：已有 distinctive_hashtags 的候選人不重複處理

輸出欄位（寫入 Candidate）：

- distinctive_hashtags: string[] (3–4 個)

對應檔案：

類型	檔案
執行指令	scripts/runStep09.ts
主要邏輯	src/services/hashtagDistinctifier.ts
Prompt	src/prompts/distinctiveHashtags.ts

指令：npm run step9 / npm run step9:mock

步驟 10 — UI 資料準備（待實作）

尚無對應檔案：步驟 10 負責將各步驟產出的欄位整理成前端可直接消費的格式（如 API response 或靜態 JSON）。目前 pipeline 已將所有結果寫回各候選人的 .json 檔，前端可直接讀取，步驟 10 的實作範圍視 UI 需求而定。

共用基礎設施

檔案	用途
src/services/gemini.ts	所有 Gemini API 呼叫（LLM + Embedding），含 retry 邏輯（最多 8 次，503 指數退避）
src/types/index.ts	全域型別定義（IdeaUnit、Candidate、StandardDictionary 等）
src/config/index.ts	讀取環境變數（OUTPUT_DIR、GEMINI_MODEL 等）

資料模型

IdeaUnit

```
{
  id: string;
  candidate_id: string;
  section: '自傳' | '讀書計畫' | '加分文件' | '其他';
  content: string;
  // Step 2:
  criteria: CriterionId[];
  sub_criteria_map: Partial<Record<CriterionId, string>>;
  hashtags: string[];
  // Step 4:
  standard_sub_criteria_map?: Partial<Record<CriterionId, string>>;
}
```

Candidate

```
{
  candidate_id: string;
  source_files: string[];
  idea_units: IdeaUnit[];
  // Step 6:
  feature_vector?: number[]; // 42 維, L2 正規化
  radar_chart_data?: Record<CriterionId, number>;
  // Step 7:
  cluster_id?: number;
  is_medoid?: boolean;
  // Step 8:
  cluster_name?: string;
  // Step 9:
  distinctive_hashtags?: string[];
}
```

ClusterSummary (clusters.json)

```
{
  algorithm: 'kmeans' | 'gmm';
  k: number;
  silhouette: number;
  clusters: Array<{
    cluster_id: number;
    size: number;
    medoid: string;
    members: string[];
    cluster_name?: string;
    top_sub_criteria?: Array<{ name: string; count: number }>;
    top_hashtags?: Array<{ tag: string; count: number }>;
  }>;
}
```

StandardDictionary (standard_dictionary.json)

```
{
  [criterionId: string]: Array<{
    id: string;           // e.g. "C1_S1"
    name: string;         // 標準 sub-criteria 名稱
    raw_labels: string[];
  }>
}
```

Output 目錄結構

```
output_mock/
├── mock_01.json           # Candidate (含所有步驟輸出欄位)
├── mock_02.json
├── ...
├── standard_dictionary.json # Step 3 產出
├── embeddings_cache.json   # Step 3 embedding 快取 (不進版控)
├── clusters.json          # Step 7+8 產出
└── umap.html              # Step 7 視覺化
```

限制事項

- **不得產生任何數值分數**供展示
- 所有分群邏輯基於向量距離與語意相似度
- LLM 呼叫不得在 prompt 中包含候選人姓名等識別資訊